



LeetCode 977 – Squares of a Sorted Array

1. Problem Title & Link

Title: LeetCode 977: Squares of a Sorted Array

Link: <https://leetcode.com/problems/squares-of-a-sorted-array/>

2. Problem Statement (Short Summary)

Given a sorted integer array `nums` (sorted in non-decreasing order), return an array containing the square of each number, also sorted in non-decreasing order.

The challenge is that negative numbers become positive after squaring and may disrupt the sorted order.

3. Examples (Input → Output)

Example 1

Input:

`nums = [-4,-1,0,3,10]`

Output:

`[0,1,9,16,100]`

Explanation:

Squares:

`[16,1,0,9,100]`

After sorting:

`[0,1,9,16,100]`

Example 2

Input:

`nums = [-7,-3,2,3,11]`

Output:

`[4,9,9,49,121]`

Example 3

Input:

`nums = [-2,-1]`

Output:

`[1,4]`

4. Constraints

`1 <= nums.length <= 104`



$-10^4 \leq \text{nums}[i] \leq 10^4$

nums is sorted in non-decreasing order.

Important:

Array is already sorted.

Need output sorted too.

5. Core Concept (Pattern / Topic)

Two Pointers

This is one of the most important introductory Two Pointer problems.

6. Thought Process (Step-by-Step Explanation)

Brute Force Approach

Idea

1. Square every element.
2. Sort the resulting array.
3. Return it.

Example:

`[-4,-1,0,3,10]`

Squares

`[16,1,0,9,100]`

Sort

`[0,1,9,16,100]`

Complexity

Squaring = $O(n)$

Sorting = $O(n \log n)$

Total = $O(n \log n)$

Works, but not optimal.



Optimized Thinking

Observe:

`[-4,-1,0,3,10]`

Largest square comes from:

$|-4| = 4$

$|10| = 10$

The largest absolute value is always at one of the ends.

Therefore:

Compare left and right

Take larger square

Place at end of answer array

Move corresponding pointer.

Key Insight

The largest square always comes from:

left end

or

right end

Hence Two Pointers.

7. Visual / Intuition Diagram (ASCII)

```
nums
```

```
[-4,-1,0,3,10]
```

```
  L           R
```

```
Squares
```

```
16           100
```

```
100 is larger
```

```
Place at last position
```

```
Answer
```

```
[_,_,__,100]
```

```
Move right pointer.
```

```
Next:
```



```
[-4,-1,0,3]
```

```
L          R
```

```
16          9
```

```
Place 16.
```

```
[_ , _ , _ , 16 , 100]
```

```
Continue.
```

8. Pseudocode

```
create answer array

left = 0
right = n-1

position = n-1

while left <= right

    leftSquare = nums[left] * nums[left]
    rightSquare = nums[right] * nums[right]

    if leftSquare > rightSquare

        answer[position] = leftSquare
        left++

    else

        answer[position] = rightSquare
        right--

    position--

return answer
```

9. Code Implementation

Python

```
class Solution:
    def sortedSquares(self, nums):

        n = len(nums)

        ans = [0] * n

        left = 0
        right = n - 1
```



```
pos = n - 1

while left <= right:

    leftSquare = nums[left] * nums[left]
    rightSquare = nums[right] * nums[right]

    if leftSquare > rightSquare:
        ans[pos] = leftSquare
        left += 1
    else:
        ans[pos] = rightSquare
        right -= 1

    pos -= 1

return ans
```

Java

```
class Solution {

    public int[] sortedSquares(int[] nums) {

        int n = nums.length;

        int[] ans = new int[n];

        int left = 0;
        int right = n - 1;

        int pos = n - 1;

        while(left <= right) {

            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if(leftSquare > rightSquare) {

                ans[pos] = leftSquare;
                left++;

            } else {

                ans[pos] = rightSquare;
                right--;

            }

            pos--;

        }

    }

}
```



```
    }  
  
    return ans;  
}  
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

Single traversal.

Space Complexity

$O(n)$

Output array.

11. Common Mistakes / Edge Cases

Mistake 1

Squaring and assuming array remains sorted.

Example:

`[-4,-1,0,3,10]`

Squares

`[16,1,0,9,100]`

Not sorted.

Mistake 2

Using sorting unnecessarily.

$O(n \log n)$

instead of

$O(n)$

Mistake 3

Filling answer from front.

The largest square comes first, so fill from end.

12. Detailed Dry Run

Input:



nums = [-4,-1,0,3,10]

Left	Right	Left ²	Right ²	Position	Answer
-4	10	16	100	4	[,,,100]
-4	3	16	9	3	[,_,16,100]
-1	3	1	9	2	[,9,16,100]
-1	0	1	0	1	[_,1,9,16,100]
0	0	0	0	0	[0,1,9,16,100]

Final Output:

[0,1,9,16,100]

13. Common Use Cases (Real-Life / Interview)

Sorted Transformations

Used when:

Apply transformation

Need sorted output

Examples:

- Data normalization
- Signal processing
- Financial analytics
- Ranking systems

14. Common Traps (Important!)

Trap 1

Thinking sorted input means sorted squares.

False.

Trap 2

Not using absolute values implicitly.

The largest square comes from the largest absolute value.

Trap 3

Wrong pointer movement.

Move only the pointer that contributed the larger square.

15. Builds To (Related LeetCode Problems)

LC 977 Squares of a Sorted Array





LC 167 Two Sum II



LC 11 Container With Most Water



LC 15 Three Sum



LC 42 Trapping Rain Water

These are core Two Pointer problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Square + Sort	$O(n \log n)$	$O(n)$	Simple
Two Pointers	$O(n)$	$O(n)$	Optimal
Tree Structure	$O(n \log n)$	$O(n)$	Overkill

Approach 1: Square + Sort

```
arr = [x*x for x in nums]
```

```
arr.sort()
```

Easy but slower.

Approach 2: Two Pointers

Compare both ends.

Best solution.

17. Why This Solution Works (Short Intuition)

The largest squared value always comes from either:

leftmost element

or

rightmost element

because those positions contain the largest absolute values.

By placing the larger square from the back of the answer array, we build the sorted result correctly in $O(n)$ time.

18. Variations / Follow-Up Questions

Follow-Up 1

Can you solve in-place?



Follow-Up 2

What if array is not sorted?

Answer:

Square + Sort

Follow-Up 3

What if we need descending order?

Fill answer from front.

Follow-Up 4

What if cube values instead of squares?

Two pointers no longer work directly because sign behavior changes.

Pattern Learned

Two Pointers

Compare both ends

Choose larger value

Move corresponding pointer

Build answer gradually